

BillingApp — Source Code Security Review

Target: `BillingApp.exe` (PyInstaller-bundled Flask application, Python 3.14) **Method:** Static analysis of recovered application logic (`app.py`, `models.py`, `pdf_generator.py`) via bytecode disassembly, since no public decompiler currently supports the Python 3.14 bytecode/marshal format. **Scope:** Application-layer code only (Flask routes, SQLAlchemy models, PDF generation). Bundled third-party libraries (Flask, SQLAlchemy, ReportLab, etc.) were excluded — those are stock packages, not custom code.

Summary

The application is a small local Flask billing/invoicing tool (SQLite + SQLAlchemy ORM, session-based auth, ReportLab PDF export). Password storage and SQL query construction are both handled safely. However, the **authentication system is effectively decorative**: every business-data endpoint is reachable with no login check at all, and the app ships with a hardcoded default admin account and a hardcoded Flask secret key. Combined, these allow full unauthenticated read/write access to all customer and billing data.

Severity	Count
Critical	2
High	3
Medium	3
Low	2

Critical Findings

C1. Broken access control — business endpoints have no authentication check

`login`, `logout`, and `get_current_user` are the only functions that reference the Flask `session` object. The following routes perform **no session/auth check whatsoever**, despite handling customer PII and billing records:

- `GET /api/bills` (`get_bills`) — list/search all bills
- `POST /api/bills` (`create_bill`) — create a bill
- `GET /api/bills/<bill_id>` (`get_bill`) — read any bill by ID
- `GET /api/bills/<bill_id>/download` (`download_bill_pdf`) — download any invoice PDF by ID

Anyone who can reach the server (including over a network if the port is ever forwarded/proxied) can list, read, create, and download all billing data without ever logging

in. This is a full IDOR + missing-authorization issue (OWASP A01: Broken Access Control) — the login system exists in the code but isn't actually enforced anywhere.

Fix: add a `@login_required`-style decorator (checking `session.get('user_id')`) to every `/api/bills*` route, and return 401 if absent.

C2. Hardcoded Flask `SECRET_KEY`

```
SECRET_KEY = 'billing-system-secret-key-2026'
```

This key signs the session cookie. Because it's a fixed string baked into every copy of the shipped executable, anyone who has the binary (i.e., every customer/user) can forge valid session cookies for any `user_id`, fully bypassing login even if C1 is fixed.

Fix: generate a random secret key per installation at first run (e.g., `secrets.token_hex(32)`) and persist it outside the binary (e.g., in the same `data` directory as the DB), never hardcode it in source.

High Findings

H1. Hardcoded default admin credentials, auto-seeded on every startup

```
username: admin
password: admin123
```

`init_db()` automatically creates this account if it doesn't already exist, on every launch. Since the credential is identical across every deployment and visible in the binary, it's a guaranteed backdoor account wherever this app is deployed.

Fix: either don't seed a default account at all, or generate a random password on first run and print/store it once for the operator to change immediately.

H2. Flask debug mode enabled

```
app.run(host='127.0.0.1', port=port, debug=True)
```

`debug=True` enables the Werkzeug interactive debugger. If any unhandled exception occurs, an attacker with access to the port gets an in-browser Python console — this is a well-known remote-code-execution vector when the debugger is reachable, and it also leaks full stack traces/source snippets on error pages. Binding to `127.0.0.1` limits this to local access under default conditions, but that's not a robust control (port forwarding, reverse proxies, or multi-user machines all defeat it).

Fix: `debug=False` in any build meant to leave a developer's machine; use proper logging instead.

H3. Weak password policy

Registration only requires a password of **4 characters or more**:

```
"Password must be at least 4 characters long"
```

This permits trivially guessable passwords (`"1234"`, `"aaaa"`), undermining the (correctly implemented) hashing underneath it.

Fix: raise the minimum (12+ recommended), and/or check against a common-password blocklist.

Medium Findings

M1. No rate limiting / brute-force protection on `/api/login`

The login route has no throttling, lockout, or delay logic. Combined with H3's weak password policy, this makes online brute-forcing of accounts (including the known-name `admin` account) practical.

Fix: add rate limiting (e.g., Flask-Limiter) and/or exponential backoff / account lockout after repeated failures.

M2. No CSRF protection on state-changing POST routes

`/api/login`, `/api/register`, `/api/logout`, and `POST /api/bills` rely purely on session cookies with no CSRF token. The CORS policy does restrict cross-origin credentialed requests to a small allowlist of localhost dev origins, which mitigates this somewhat in the current dev-style config — but if that origin list is ever loosened for a production deployment, these routes become CSRF-exploitable.

Fix: add CSRF tokens (e.g., `flask-wtf` CSRF protection) to all state-changing routes regardless of CORS config, since CORS and CSRF protect against different things.

M3. Verbose error messages returned to the client

`download_bill_pdf` and `create_bill` both return raw exception text to the caller on failure (e.g., `f"Failed to generate PDF: {str(e)}"`). This can leak internal file paths, library internals, or other implementation details useful for further attacks.

Fix: log full exception details server-side only; return a generic error message to the client.

Low Findings

L1. CORS origins hardcoded to localhost dev URLs

```
python
```

```
CORS(app, supports_credentials=True, origins=[
    'http://localhost:5173', 'http://127.0.0.1:5173',
    'http://localhost:5000', 'http://127.0.0.1:5000'
])
```

Not a vulnerability as shipped (this is a reasonably tight allowlist), but worth flagging so it isn't silently widened (e.g., to `*`) in a future build — `supports_credentials=True` combined with a wildcard origin would be a serious CORS misconfiguration.

L2. SQLite database and generated invoice PDFs stored unencrypted next to the executable

`billing.db` and generated invoice PDFs live in a `data/` folder beside the binary with no encryption at rest. For an app that explicitly stores customer PII (name, email, phone, address) and billing totals, this means anyone with filesystem access to the machine (or a copy of the install folder) has full access to all records, independent of the application's own auth layer.

Fix: consider OS-level disk encryption as a baseline, and/or encrypting sensitive DB fields if this app is meant to run on shared or less-trusted machines.

What was done correctly

- **Password storage:** uses `werkzeug.security.generate_password_hash`/`check_password_hash` — proper salted hashing, not plaintext or a weak fast hash.
- **SQL injection:** all database access goes through SQLAlchemy's ORM query API (`filter_by`, `ilike`, etc.) with no raw/string-interpolated SQL anywhere in the codebase.
- **No use of `eval`, `exec`, or `pickle`** anywhere in the reviewed code — a common source of RCE in Python apps, and it's absent here.
- **Static file serving** (`serve_frontend`) uses Flask's `send_from_directory`, which internally guards against path traversal (`..`) in modern Werkzeug versions.

Priority Remediation Order

1. Add authentication checks to all `/api/bills*` routes (C1)
 2. Replace hardcoded `SECRET_KEY` with a per-install random value (C2)
 3. Remove/randomize the default `admin/admin123` account (H1)
 4. Disable Flask debug mode (H2)
 5. Strengthen password policy + add login rate limiting (H3, M1)
 6. Add CSRF protection to state-changing routes (M2)
 7. Sanitize error responses returned to clients (M3)
-

Note: this review is based on a disassembly of the compiled bytecode rather than the original source, since no public decompiler currently supports Python 3.14.

Variable/function names were preserved by the compiler and match what's shown above; only inline comments (if any existed) were not recoverable.